

EXPERIMENT – 06(HDOC)

C Programming

Question 1: Moment of Inertia of a Ring

Step 1: Understand the Problem Statement

- Input: Mass (M) of the ring and Radius (R) of the ring.
- Output: Moment of inertia (I) of the ring about its central axis.
- The moment of inertia of a ring depends on how its mass is distributed at a fixed radius from the axis of rotation.

Formula used: $I = M \times R^2$ (I = moment of inertia, M = mass, R = radius)

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	Mass = 2 kg Radius = 3 m	18.0000 kg.m ²	18.0000 kg.m ²	Pass
2	Mass = 5 kg Radius = 0 m	0.0000 kg.m ²	0.0000 kg.m ²	Pass
3	Mass = 1.5 kg Radius = 9.3750 m	131.8359 kg.m ²	131.8359 kg.m ²	Pass

Step 3: Algorithm

1. Start the program.
2. Declare variables mass, radius and inertia as floating point numbers.
3. Read the value of mass (M) from the user.
4. Read the value of radius (R) from the user.
5. Compute inertia = mass × radius × radius.
6. Display the computed value of inertia.
7. Stop the program.

Step 4: C Program

```

/* Q1: Moment of inertia of a ring given mass and radius
   Formula: I = M * R^2
*/
#include <stdio.h>

int main() {
    float mass, radius, inertia;

```

```

printf("Enter mass of the ring (kg): ");
scanf("%f", &mass);
printf("Enter radius of the ring (m): ");
scanf("%f", &radius);

// Input Validation
if (mass <= 0 || radius <= 0) {
    printf("Error: Mass and radius must be positive.\n");
    return 1;
}

inertia = mass * radius * radius;

printf("Moment of Inertia of the ring = %.4f kg.m^2\n", inertia);

return 0;
}

```

Step 5: Compile, Execute and Evaluate

The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

```
cc hdoc6.c -o a.out      (compile)
```

```
./a.out                  (execute)
```

All 3 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot

```

(kali㉿kali)-[~]
$ nano hdoc6.c

(kali㉿kali)-[~]
$ cc hdoc6.c

(kali㉿kali)-[~]
$ ./a.out
Enter mass of the ring (kg): 2
Enter radius of the ring (m): 3
Moment of Inertia of the ring = 18.0000 kg.m^2

(kali㉿kali)-[~]
$ ./a.out
Enter mass of the ring (kg): 5
Enter radius of the ring (m): 0
Error: Mass and radius must be positive.

(kali㉿kali)-[~]
$ ./a.out
Enter mass of the ring (kg): 1.5
Enter radius of the ring (m): 9.3750
Moment of Inertia of the ring = 131.8359 kg.m^2

```

Question 2: Fluid Speed using Continuity Equation

Step 1: Understand the Problem Statement

- Input: Cross-sectional area A1, cross-sectional area A2, and fluid speed V1 at area A1.
- Output: Fluid speed V2 at cross-sectional area A2.
- The continuity equation states that for an incompressible fluid in steady flow, the mass flow rate is the same at every cross-section of the pipe.

Formula used: $A1 \times V1 = A2 \times V2 \Rightarrow V2 = (A1 \times V1) / A2$

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	A1=2, A2=4, V1=8	V2 = 4.0000 m/s	V2 = 4.0000 m/s	Pass
2	A1=5, A2=5, V1=10	V2 = 10.0000 m/s	V2 = 10.0000 m/s	Pass
3	A1=3, A2=1, V1=2	V2 = 6.0000 m/s	V2 = 6.0000 m/s	Pass

Step 3: Algorithm

1. Start the program.
2. Declare variables A1, A2, V1 and V2 as floating point numbers.
3. Read the value of area A1 from the user.
4. Read the value of area A2 from the user.
5. Read the value of speed V1 from the user.
6. Check that A2 is not zero (to avoid division by zero).
7. Compute $V2 = (A1 \times V1) / A2$.
8. Display the computed value of V2.
9. Stop the program.

Step 4: C Program

```
/* Q2: Fluid speed using continuity equation
   Continuity equation: A1 * V1 = A2 * V2
   Given: A1, A2, V1  -> Find V2
*/
#include <stdio.h>

int main() {
    float A1, A2, V1, V2;

    printf("Enter cross-sectional area A1 (m^2): ");
    scanf("%f", &A1);
```

```

printf("Enter cross-sectional area A2 (m^2): ");
scanf("%f", &A2);
printf("Enter fluid speed V1 at area A1 (m/s): ");
scanf("%f", &V1);

// Input Validation
if (A1 <= 0 || A2 <= 0 || V1 <= 0) {
    printf("Error: Areas and speed must be positive.\n");
    return 1;
}

V2 = (A1 * V1) / A2;

printf("Fluid speed V2 at area A2 = %.4f m/s\n", V2);

return 0;
}

```

Step 5: Compile, Execute and Evaluate

The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

```
cc hdoc6_2.c      (compile)
```

```
./a.out          (execute)
```

All 3 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2)

Step 6: Compilation and Execution Screenshot

```

(kali@kali)-[~]
$ nano hdoc6_2.c
(kali@kali)-[~]
$ cc hdoc6_2.c
(kali@kali)-[~]
$ ./a.out
Enter cross-sectional area A1 (m^2): 2
Enter cross-sectional area A2 (m^2): 4
Enter fluid speed V1 at area A1 (m/s): 8
Fluid speed V2 at area A2 = 4.0000 m/s
(kali@kali)-[~]
$ ./a.out
Enter cross-sectional area A1 (m^2): 5
Enter cross-sectional area A2 (m^2): 5
Enter fluid speed V1 at area A1 (m/s): 10
Fluid speed V2 at area A2 = 10.0000 m/s
(kali@kali)-[~]
$ ./a.out
Enter cross-sectional area A1 (m^2): 3
Enter cross-sectional area A2 (m^2): 1
Enter fluid speed V1 at area A1 (m/s): 2
Fluid speed V2 at area A2 = 6.0000 m/s

```

Question 3: Apparent Depth given Real Depth and Refractive Index

Step 1: Understand the Problem Statement

- Input: Real depth of the object and refractive index of the medium.
- Output: Apparent depth of the object as seen from outside the medium.
- When viewed from a rarer medium, an object placed inside a denser medium appears to be raised, i.e. its apparent depth is less than its real depth.

Formula used: $\text{Apparent Depth} = \text{Real Depth} / \text{Refractive Index}$

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	Real depth=10, n=1.33	7.5188 m	7.5188 m	Pass
2	Real depth=20, n=1.5	13.3333 m	13.3333 m	Pass
3	Real depth=15, n=1	15.0000 m	15.0000 m	Pass

Step 3: Algorithm

1. Start the program.
2. Declare variables realDepth, refractiveIndex and apparentDepth as floating point numbers.
3. Read the value of real depth from the user.
4. Read the value of refractive index from the user.
5. Check that refractive index is not zero (to avoid division by zero).
6. Compute $\text{apparentDepth} = \text{realDepth} / \text{refractiveIndex}$.
7. Display the computed value of apparentDepth.
8. Stop the program.

Step 4: C Program

```
/* Q3: Apparent depth given real depth and refractive index
   Formula: Apparent Depth = Real Depth / Refractive Index
*/
#include <stdio.h>

int main() {
    float realDepth, refractiveIndex, apparentDepth;
```

```

printf("Enter real depth (m): ");
scanf("%f", &realDepth);
printf("Enter refractive index of the medium: ");
scanf("%f", &refractiveIndex);

// Input Validation
if (realDepth <= 0 || refractiveIndex < 1.0) {
    printf("Error: Depth must be positive and refractive index >= 1.0.\n");
    return 1;
}

apparentDepth = realDepth / refractiveIndex;

printf("Apparent Depth = %.4f m\n", apparentDepth);

return 0;
}

```

Step 5: Compile, Execute and Evaluate

The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

<code>cc hdoc6_3.c -o a.out</code>	(compile)
<code>./a.out</code>	(execute)

All 3 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot

```

kali@kali:~$ nano hdoc6_3.c
kali@kali:~$ cc hdoc6_3.c
kali@kali:~$ ./a.out
Enter real depth (m): 10
Enter refractive index of the medium: 1.33
Apparent Depth = 7.5188 m
kali@kali:~$ ./a.out
Enter real depth (m): 20
Enter refractive index of the medium: 1.5
Apparent Depth = 13.3333 m
kali@kali:~$ ./a.out
Enter real depth (m): 15
Enter refractive index of the medium: 1
Apparent Depth = 15.0000 m

```

❖ Conclusion:

All three C programs for calculating the Moment of Inertia, Fluid Speed using the Continuity Equation, and Apparent Depth were successfully implemented, compiled, and executed. The actual outputs obtained from the terminal execution matched the expected theoretical results across all test cases, confirming the correctness and robustness of the code logic.
